

GreenLight AI

Manual Técnico

Adrián Pavón Alcón

Antonio Guisado Marin

Proyecto Intermodular · Máster de Inteligencia Artificial y Big Data

Centro FP Superior · Campus Cámara Comercio Sevilla

Mayo 2026

Guía de instalación, configuración y desarrollo · v2.1.0

Índice de Contenidos

1. Estructura del repositorio

2. Instalación y configuración

2.1 Requisitos previos

2.2 Instalación

2.3 Configuración de la API key de Mistral

2.4 Ejecución

3. Dependencias

4. Arquitectura del código

4.1 src/mistral_definitivo.py

4.2 src/dataset_final.py

4.3 ui/app.py

4.4 ui/generar_informe.py

5. Integrar el modelo .pkl en la UI

6. Despliegue en Streamlit Cloud

7. Scripts de instalación y arranque (.bat)

7.1 setup_env.bat

7.2 run_app.bat

7.3 Flujo de uso completo

7.4 Ubicación en el repositorio

1. Estructura del repositorio

El repositorio sigue una estructura modular. Cada carpeta tiene una responsabilidad clara y no mezcla responsabilidades:

Ruta	Descripción
catboost_info/	Logs generados automáticamente por CatBoost durante el entrenamiento
data/dataset_sintetico_soccermon.csv	Dataset sintético de 30.000 registros (3.3 MB)
data/historial_analisis.csv	Registro histórico de análisis realizados con la aplicación
docs/01_Memoria_GreenLight_AI.docx	Memoria del Proyecto Intermodular (este documento)
docs/02_Evaluacion_Tecnica.docx	Evaluación técnica del sistema: métricas, casos de prueba, comparativas
docs/03_Manual_Tecnico.docx	Manual técnico: instalación, configuración, arquitectura, despliegue
docs/04_Manual_Usuario.docx	Manual de usuario: guía completa para el cuerpo técnico
models/*.pkl	7 modelos entrenados y serializados: CatBoost, LightGBM, RF, LR, XGB, Stacking, scaler
reports/*.png	Gráficos generados durante el entrenamiento: SHAP, ROC, correlaciones, boxplots
reports/logo_greenlight.jpg	Logo oficial de GreenLight AI
src/dataset_final.py	Pipeline completo de generación del dataset y entrenamiento del ensemble
src/mistral_definitivo.py	Lógica completa del DSS: NLP, sistema experto, reglas deterministas
ui/app.py	Interfaz Streamlit con las 4 pantallas de la aplicación
ui/generar_informe.py	Generador de informes PDF con diseño oscuro profesional (ReportLab)
requirements.txt	Dependencias Python del proyecto con versiones mínimas
Arquitectura_definitiva.png	Diagrama oficial de la arquitectura del sistema
README.md	Descripción general del repositorio
setup_env.bat	Script de instalación del entorno virtual, dependencias y configuración de la API key de Mistral (secrets.toml). Ejecutar solo una vez.
run_app.bat	Script de arranque de la aplicación. Verifica el entorno virtual y el secrets.toml, lanza streamlit run ui/app.py y abre la app en localhost:8501.

2. Instalación y configuración

2.1 Requisitos previos

- Python 3.10 o superior
- pip actualizado (pip install --upgrade pip)
- Cuenta en Mistral AI con API key (plan Experiment gratuito)
- Git para clonar el repositorio

2.2 Instalación

Clonar el repositorio e instalar dependencias:

```
git clone https://github.com/[usuario]/ProyectoIntermodular.gitcd ProyectoIntermodularpip install -r requirements.txt
```

2.3 Configuración de la API key de Mistral

La API key nunca debe escribirse en el código ni subirse al repositorio. Streamlit usa un archivo de secretos que está excluido del .gitignore:

```
# Crear el archivo .streamlit/secrets.tomlmkdir .streamlitecho 'MISTRAL_API_KEY =  
"tu_api_key_aqui"' > .streamlit/secrets.toml
```

2.4 Ejecución

```
cd uistreamlit run app.py
```

La aplicación abre automáticamente en <http://localhost:8501>

3. Dependencias

Librería	Versión mínima	Uso en el proyecto
streamlit	≥1.35.0	Interfaz web completa
pandas	≥2.0.0	Manipulación de datos y CSV
numpy	≥1.26.0	Operaciones numéricas
scikit-learn	≥1.4.0	Stacking, RobustScaler, métricas
joblib	≥1.3.0	Serialización de modelos .pkl
xgboost	≥2.0.0	Modelo base XGBoost
lightgbm	≥4.3.0	Modelo base LightGBM
catboost	≥1.2.0	Modelo base CatBoost
plotly	≥5.20.0	Gráficos interactivos (gauge, radar, bar)
pdfplumber	≥0.11.0	Extracción de texto de PDFs subidos
reportlab	≥4.0.0	Generación de informes PDF
pypdf	≥4.0.0	Procesamiento PDF auxiliar
requests	≥2.31.0	Llamadas HTTP a la API de Mistral
mistralai	≥1.0.0	SDK oficial de Mistral

4. Arquitectura del código

4.1 src/mistral_definitivo.py

Es el backend principal del sistema. Contiene toda la lógica del DSS, el módulo NLP y el sistema experto. Sus funciones principales son:

Función	Descripción
llamar_modelo(prompt, max_tokens, temp)	Llamada HTTP robusta a Mistral con reintentos, gestión rate limit 429 y medición de latencia.
call_llm_json(prompt, schema)	Obtiene respuesta JSON validada. Si falla, repara automáticamente con un segundo prompt.
extraer_json_seguro(texto)	Extrae JSON de texto libre usando regex. Doble intento (regex + slice).
validar_features(f)	Normaliza y valida el JSON de extracción. Clamp de valores a rangos válidos.
validar_decision(d)	Valida que la decisión del LLM es ROJO, ÁMBAR o VERDE.
normalizar_zona(zona)	Mapea cualquier descripción textual a la clave canónica del diccionario.
extraer_caracteristicas_cualitativas(reporte)	Orquesta NLP: Mistral o fallback heurístico según disponibilidad y confianza.
HeuristicParser.parse(texto)	Parser basado en palabras clave. Actúa como fallback cuando Mistral no está disponible.
obtener_tiempo_recuperacion_determinista(crit, sev)	Fallback de tiempo de baja por criticidad × severidad numérica.
decision_arbitrada(ml_prob, features, reporte, acwr, use_llm, posicion)	Función principal. Ejecuta las 5 capas de decisión y devuelve el resultado final.

4.2 src/dataset_final.py

Pipeline completo de generación del dataset sintético y entrenamiento del ensemble. Al ejecutarse genera el dataset, entrena los 6 modelos, los serializa en models/ y produce los gráficos en reports/.

4.3 ui/app.py

Interfaz Streamlit con 4 pantallas. La comunicación con el backend se hace importando las funciones de src/mistral_definitivo.py. La probabilidad ML actualmente usa una fórmula compuesta; la integración del modelo .pkl es el siguiente paso de desarrollo.

4.4 ui/generar_informe.py

Genera informes PDF con diseño oscuro profesional usando ReportLab. Incluye dos custom Flowables: Semaforo (dibuja el semáforo con glow) y BarraScore (barra de progreso visual del score de riesgo).

5. Integrar el modelo .pkl en la UI

El único pendiente técnico relevante es conectar el modelo_stacking_ensemble.pkl en app.py. El modelo ya existe en models/ y está listo. El código a añadir en la sección del Analizador Híbrido es:

```
import joblibROOT_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))scaler = joblib.load(os.path.join(ROOT_DIR, 'models', 'scaler.pkl'))model = joblib.load(os.path.join(ROOT_DIR, 'models', 'modelo_stacking_ensemble.pkl'))# Construir X con las 13 features EN EL ORDEN EXACTO del entrenamiento:# distance_m, hsr_distance_m, acwr, ctl, atl, fatigue_score, sleep_quality,# stress, muscle_soreness, inter_acwr_fatigue, th_acwr_fatigue,# th_stress_sleep, inter_sleep_soreX = np.array([[dist, dist_hd, acwr_val, ctl, atl, fatiga_subj, horas_sueno, estres, dolor, inter_acwr_fatigue, th_acwr_fatigue, th_stress_sleep, inter_sleep_sore]])X_scaled = scaler.transform(X)prob_ml = float(model.predict_proba(X_scaled)[0, 1])
```

Las variables ctl, atl, estres y dolor no tienen input en la UI actual. Se puede añadir un bloque desplegable con valores avanzados o usar valores por defecto (ctl=50, atl=70, estres=5, dolor=3).

6. Despliegue en Streamlit Cloud

1. Subir el repositorio a GitHub (asegurarse de que secrets.toml está en .gitignore).
2. Acceder a share.streamlit.io y conectar el repositorio.
3. Configurar la API key de Mistral en Secrets de Streamlit Cloud (interfaz web).
4. Definir el Main file path como ui/app.py.
5. Desplegar. Streamlit Cloud instala requirements.txt automáticamente.

Los modelos .pkl grandes (Random Forest: 36.7 MB, Stacking: 78.3 MB) pueden superar el límite de 100 MB de GitHub LFS. Considerar almacenar en Google Drive o Hugging Face Hub y cargarlos dinámicamente al arrancar la app.

7. Scripts de instalación y arranque (.bat)

El repositorio incluye dos scripts de Windows (.bat) en la raíz del proyecto que automatizan la instalación del entorno y el arranque de la aplicación. Ambos archivos deben colocarse al mismo nivel que requirements.txt para que las rutas relativas funcionen correctamente.

7.1 setup_env.bat

Ejecutar una sola vez para preparar el entorno completo. Realiza los siguientes pasos en orden:

Paso 1 — Verificación de Python 3.10

Comprueba que Python 3.10 está instalado usando el launcher py. Si no lo encuentra, detiene la ejecución e indica la URL de descarga.

Paso 2 — Creación del entorno virtual

Crea un entorno virtual aislado en la carpeta .venv. Si ya existe, lo reutiliza sin sobrescribirlo.

Paso 3 — Actualización de pip

Actualiza pip dentro del entorno virtual para garantizar compatibilidad con las versiones mínimas del requirements.txt.

Paso 4 — Instalación de dependencias

Instala todas las librerías del requirements.txt dentro del entorno virtual. Si falla, detiene la ejecución e indica que se revise la conexión a internet.

Paso 5 — Configuración de la API key de Mistral (secrets.toml)

Este paso automatiza la creación del archivo .streamlit/secrets.toml, que es la forma en que Streamlit expone las variables secretas al código. El proyecto lee la API key con la instrucción:

```
| MISTRAL_API_KEY = st.secrets.get('MISTRAL_API_KEY', None)
```

La lógica del script en este paso es la siguiente:

Si el archivo no existe: el script pregunta en consola "Introduce tu MISTRAL_API_KEY (ENTER para omitir)". Si el usuario escribe la key, la guarda en el formato exacto que lee Streamlit (MISTRAL_API_KEY = "la_key"). Si pulsa ENTER sin escribir nada, crea el archivo con la key vacía y avisa que el sistema funcionará en modo heurístico sin LLM.

Si el archivo ya existe: no lo sobrescribe. Informa al usuario de que puede editarlo manualmente si necesita cambiar la key.

El archivo .streamlit/secrets.toml está excluido del repositorio por el .gitignore. La API key nunca se sube a GitHub. Si el repositorio se clona en otro equipo, el usuario deberá ejecutar de nuevo setup_env.bat para configurar su propia key.

7.2 run_app.bat

Script de arranque de la aplicación. Se ejecuta cada vez que se quiera iniciar GreenLight AI tras la instalación inicial. Realiza tres comprobaciones antes de lanzar Streamlit:

Verificación del entorno virtual: si .venv no existe, detiene la ejecución e indica que hay que ejecutar primero setup_env.bat.

Verificación del secrets.toml: si el archivo no existe, avisa pero no bloquea. La aplicación arranca en modo heurístico sin LLM.

Arranque: lanza streamlit run ui\app.py desde el entorno virtual. La aplicación se abre automáticamente en <http://localhost:8501>. Para detenerla se usa Ctrl+C en la ventana de consola.

7.3 Flujo de uso completo

El flujo completo para un usuario nuevo que instala el proyecto por primera vez es el siguiente:

```
# 1. Clonar el repositorio
git clone https://github.com/[usuario]/ProyectoIntermodular.git
cd ProyectoIntermodular
# 2. Instalar entorno (solo la primera vez)
setup_env.bat
# 3. Arrancar la aplicación (cada vez que se use)
run_app.bat
```

7.4 Ubicación en el repositorio

Ambos archivos deben situarse en la raíz del repositorio, al mismo nivel que requirements.txt. Desde ahí tienen acceso directo a requirements.txt para la instalación, a la carpeta .streamlit/ para el secrets.toml y a ui\app.py para el arranque de Streamlit:

```
ProyectoIntermodular/
├── setup_env.bat      ← instalar (solo 1 vez)
├── run_app.bat       ← arrancar la app
├── requirements.txt
├── ui/app.py
└── .streamlit/secrets.toml ← generado por setup_env.bat
```